Dimension Reduction with Latent Variable Models for Simplification of Image Data

Joey Hotz

University of Toronto, Department of Statistical Sciences

April 03, 2026

Team Information: All aspects of this project (coding, writing the report, creating slides, project motivation, etc.) were done by Joey Hotz.

All code is available on GitHub: https://github.com/hotz1/STA2201-Final-Project

1 Introduction

The overarching goal of this project is to investigate the properties of Probabilistic Principal Component Analysis (PPCA) and Gaussian Process Latent Variable Models (GP-LVM), with a focus on the application of these methods in dimension reduction on an image dataset

In this report, we discuss some of the statistical background which underpins each of these methods and we describe some of the major formulas required for using these methods in a practical setting. Following this, we examine the performance of each of these methods on the `Fashion-MNIST` dataset (Xiao et al., 2017) to better understand the differences and similarities between these latent variable methods.

The primary research question which motivates this research project is to find models which are suitable for reducing the dimensionality of image data. Using the `Fashion-MNIST` dataset as our source of image data with corresponding classification labels, we measure the performance of these methods using two metrics:

- ★ The average reconstruction error for reconstructing the original images from the fitted latent variables.
- ★ The agreement between clusters formed in the lower-dimensional space with the true labels for the respective images.

In addition to these two quantifiable metrics, we also use dimension reduction methods to create 2-D and 3-D plots of the dataset, which provides valuable qualitative information about a hard-to-visualize data distribution.

My personal motivation for choosing this topic was to gain a better understanding of some dimension reduction methods which we did not discuss in much detail during the past semester. I am particularly interested in the aspects of this research project which relate to the GP-LVM methodology, as we did not cover these methods of performing non-linear dimension reduction during STA2201, and this project allowed me to research these particular methods in much greater detail.

2 Methodology

In order to discuss the specific differences between the 'deterministic' PCA, probabilistic PCA, and Gaussian Process Latent Variable Models, we must first describe each of these methods from a statistical perspective. These methods build upon one another, with probabilistic PCA being an extension of PCA, with GP-LVM further extending this probabilistic formulation into a non-linear setting.

We first briefly describe the standard Principal Component Analysis algorithm, and we then extend this discussion to the other two methodologies which are the primary focus of this report. The dimensionality reduction setting which we use each of these three methods for can be described by the following. Suppose that we have a data matrix $\mathbf{Y} \in \mathbb{R}^{N \times D}$, where each row of $\mathbf{Y}$ is a vector $\mathbf{y}_i^\intercal \in \mathbb{R}^D$. To avoid some of the common pitfalls which may arise when working with high-dimensional data, we may be inclined to try and reduce the dimensionality of this data to make the observations more manageable. Conceptually, we can think of dimension reduction methods as invoking a mapping of the form $\phi : \mathbb{R}^D \to \mathbb{R}^Q$ where $Q < D$ (and possibly $Q \ll D$) so that we can instead focus on the transformed observations $\phi(\mathbf{y}_i) \in \mathbb{R}^Q$, which are less prone to the issues that plague high-dimensional statistics.

2.1 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is one of the most commonly-applied methods for dimensionality reduction (Tipping and Bishop, 1999). In order to effectively apply PCA for the purpose of dimension reduction, we must assume that the observations $\mathbf{y}_1, \ldots, \mathbf{y}_N$ can be represented to a sufficient degree by some lower-dimensional hyperplane.

Fundamentally, PCA relies on projecting each observation $\mathbf{y}_i \in \mathbb{R}^D$ to some r-dimensional hyperplane embedded within $\mathbb{R}^D$ such that a majority of the information in the design matrix $\mathbf{Y}$ is retained after applying this lower-dimensional projection operation. From a conceptual perspective, maximizing the retained information can be thought of in terms of minimizing the amount of information lost by the projection of every observation onto a lower-dimensional subspace, which consequently minimizes the reconstruction error when comparing the original vectors $\mathbf{y}_i$ to their projected complements which live on the lower-dimensional hyperplane.

2.2 Probabilistic PCA

Probabilistic Principal Component Analysis (PPCA) is an extension of the 'deterministic' PCA framework which formulates the dimension reduction into a lower-dimensional subspace by relating this dimension reduction to unobserved latent variables in a statistical model. (Tipping and Bishop, 1999; Lawrence, 2004)

The statistical framework for PPCA assumes that each vector of observations $\mathbf{y}_i \in \mathbb{R}^D$ can be represented by some affine linear equation of the form

$$\mathbf{y}_i = \mathbf{W}\mathbf{u}_i + \boldsymbol{\mu} + \boldsymbol{\epsilon}_i \tag{2.1}$$

Here, $\mathbf{u}_i \in \mathbb{R}^Q$ is a latent Q-dimensional vector which corresponds to $\mathbf{y}_i$ and $\boldsymbol{\epsilon}_i \sim \mathcal{N}_D(\mathbf{0}, \boldsymbol{\Psi})$ is an error term which follows a multivariate Gaussian distribution. The parameters $\mathbf{W} \in \mathbb{R}^{D\times Q}$ and $\boldsymbol{\mu} \in \mathbb{R}^D$ in Equation 2.1 are deterministic parameters which describe the affine linear transformation relating a latent unobserved vector $\mathbf{u} \in \mathbb{R}^Q$ to the corresponding observation vector $\mathbf{y} \in \mathbb{R}^D$.

To motivate the probabilistic PCA method, we make two simplifying assumptions regarding the distributions of the underlying variables:

- ★ The latent vector $\mathbf{u} \in \mathbb{R}^Q$ is a random variable with distribution $\mathbf{u} \sim \mathcal{N}_Q(\mathbf{0}, \mathbf{I}_Q)$.
- ★ $\mathrm{Cov}(\boldsymbol{\epsilon}) = \boldsymbol{\Psi} = \sigma^2 \mathbf{I}_D$ for some $\sigma^2 > 0$.

Under the two assumptions above, we have the following distributional results:

$$\mathbf{y}|\mathbf{u} \sim \mathcal{N}_D\big(\mathbf{W}\mathbf{u} + \boldsymbol{\mu}, \sigma^2 \mathbf{I}_D\big) \tag{2.2}$$

Integrating Equation 2.2 with respect to the distribution of $\mathbf{u}$ yields

$$\mathbf{y} \sim \mathcal{N}_D\big(\boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\mathsf{T} + \sigma^2 \mathbf{I}_D\big) \tag{2.3}$$

Under the PPCA model with the assumptions above, we can derive a maximum likelihood estimator for the 'shifting' vector $\boldsymbol{\mu} \in \mathbb{R}^D$, given by $\boldsymbol{\mu}_{\mathrm{MLE}} = \frac{1}{N}\sum_{i=1}^N \mathbf{y}_i$, and we can similarly derive an estimator $\mathbf{S}_{\mathrm{MLE}} = \frac{1}{N}\sum_{i=1}^N (\mathbf{y}_i - \boldsymbol{\mu}_{\mathrm{MLE}})(\mathbf{y}_i - \boldsymbol{\mu}_{\mathrm{MLE}})^\mathsf{T}$ as an estimator of the covariance matrix.

However, under this particular set of assumptions, we can also derive *exact* closed-form equations for the maximum likelihood estimators of $\mathbf{W}$ and σ^2, which typically can only be found using iterative methods such as the Expectation-Maximization algorithm. (Tipping and Bishop, 1999; Lawrence, 2005) Under these conditions, the PPCA formulation provides the following maximum likelihood estimators:

$$\mathbf{W}_{\mathrm{MLE}} = \mathbf{U}_Q \left(\boldsymbol{\Lambda}_Q - \sigma^2 \mathbf{I}_Q\right)^{1/2} \mathbf{R} \tag{2.4}$$

$$\sigma^2_{\mathrm{MLE}} = \frac{1}{D-Q} \sum_{j=Q+1}^{D} \lambda_j \tag{2.5}$$

In Equation 2.4, the matrices $\mathbf{U}_Q \in \mathbb{R}^{D\times Q}$ and $\boldsymbol{\Lambda}_Q \in \mathbb{R}^{Q\times Q}$ are comprised of the top Q eigenvectors and eigenvalues of the covariance matrix $\mathbf{S}_{\mathrm{MLE}}$. The matrix $\mathbf{R} \in \mathbb{R}^{Q\times Q}$ is an arbitrary orthogonal rotation matrix, as the likelihood function for PPCA is constant with respect to orthogonal rotations of the latent space. This formula for $\mathbf{W}_{\mathrm{MLE}}$ aligns nicely

with the non-probabilistic formulation of PCA, in which the Q principal components are derived from the top Q eigenvectors of the sample covariance matrix.

Tipping and Bishop (1999) provides a helpful way to interpret Equation 2.5, by considering the summation over the remaining $(D - Q)$ eigenvalues of $\mathbf{S}_{\text{MLE}}$ to be the variance in the data which is 'lost' (i.e. not explained) after projection onto the lower-dimensional subspace.

2.3 Gaussian Processes

Before discussing the GP-LVM methodology in full, we first describe the statistical formulation of Gaussian Processes, as Gaussian Processes and Gaussian Process regression are essential aspects of the GP-LVM framework.

A Gaussian Process $\{X_t\}_{t \in T}$ is a stochastic process of random variables $\{X_t\}$ for t in some pre-determined indexing set T which has the property that the joint distribution of any any *finite* collection of random variables from this process will be a multivariate Gaussian distribution (Rasmussen and Williams, 2006; Lawrence, 2005). Rewriting the above in more formal notation, for any set of indices $\{t_1, \ldots, t_n\} \subset T$, the n-dimensional random vector $\begin{pmatrix} X_{t_1} & \cdots & X_{t_n} \end{pmatrix}^{\mathsf{T}} \in \mathbb{R}^n$ has a multivariate Gaussian distribution. Note that as a clear corollary of this definition, any individual random variable X_t must follow a univariate Gaussian distribution.

From a statistical perspective, a Gaussian Process defined on the indexing set T is entirely specified by two functions; a *mean function* $\mu : T \to \mathbb{R}$ which represents the expectation of the random variables and a *covariance kernel function* $k : T \times T \to \mathbb{R}$, which represents the covariance between the random variables. In particular, we have that $\mu(t) = \mathbb{E}[X_t]$ and $k(t, t') = \text{Cov}(X_t, X_{t'})$.

Gaussian Processes are commonly used in regression as a 'surrogate model' for modelling unknown functions, as these models are very flexible and capable of representing many different classes of real-valued functions. In addition, Gaussian processes are very amenable to being formulated as an unknown real-valued function on a specified domain, and certain constraints such as continuity and differentiability can be imposed through specific choices of the kernel function $k(\cdot, \cdot)$ (Titsias and Lawrence, 2010).

In such regression contexts, we are given a set of inputs $\mathbf{X}_{\text{obs}} = \{\mathbf{x}_i\}_{i=1}^N \in \mathbb{R}^{N \times D}$ and $\mathbf{Y}_{\text{obs}} = \{y_i\}_{i=1}^N \in \mathbb{R}^N$, where it is assumed that $y_i = f(\mathbf{x}_i) + \epsilon_i$ for some unknown function $f : \mathbb{R}^D \to \mathbb{R}$, with ϵ_i representing some additional observation noise that comes from a Gaussian distribution. The goal of these Gaussian process regression methods is to learn the function $\mu(t) = \mathbb{E}[X_t]$ based on the given data.

In practice, the covariance kernel k is typically not *fully* specified a priori. Instead, in many Gaussian Process regression settings, the covariance kernel function k is selected to be a member of some parametric family of kernel functions. During each iteration of the model-fitting process, the hyperparameters for the specific kernel function k are updated via empirical Bayes updates to maximize the joint likelihood over the hyperparameters of the Gaussian process surrogate model and the data (Rasmussen and Williams, 2006).

2.4 Gaussian Process Latent Variable Models

Now that we have discussed the key ideas behind Gaussian Process regression, we can focus on the GP-LVM framework. Similar to both versions of PCA discussed previously, this method is primarily used for reducing the dimensionality of observed data. However, unlike the two formulations of PCA, this method does not assume that the relationship between the observations $\{\mathbf{y}_i\}_{i=1}^N$ and the latent variables $\{\phi(\mathbf{y}_i)\}_{i=1}^N$ is necessarily linear. This relaxation of the assumptions is what motivates the use of Gaussian processes to assist in learning the underlying functional forms.

In the 'typical' Gaussian process regression methodology, we learn the function $\mu(t) = \mathbb{E}[X_t]$ based on a set of *known* function inputs $\mathbf{x}_i$ and noisily-observed function outputs $\mathbf{y}_i$. In such settings, we can think of the outputs $\mathbf{y}_i$ as $\mu(\mathbf{x}_i) + \boldsymbol{\epsilon}_i$, representing the true mean with some added noise. However, for the GP-LVM method, we instead treat the observed vectors $\mathbf{y} \in \mathbb{R}^D$ as the outputs of Gaussian processes, and the inputs to these functions are unknown. In a more mathematical form, we have that

$$\mathbf{y} = g(\mathbf{x}) + \boldsymbol{\epsilon} \tag{2.6}$$

In Equation 2.6, there is an unknown function $g : \mathbb{R}^Q \to \mathbb{R}^D$ which must be learned, in addition to learning the true latent variables $\mathbf{x}$ which correspond to these outputs. This is a reversal of the standard function regression setting, as instead of learning a 'forward' mapping $\mathbf{X} \longrightarrow \mathbf{Y}$, we regress on a 'backwards' mapping $\mathbf{X} \longleftarrow \mathbf{Y}$. (Lawrence, 2004; Titsias and Lawrence, 2010)

As a simplification, the functional form of $g(\cdot)$ in Equation 2.6 is typically assumed to be comprised of D independent functions $g_d : \mathbb{R}^Q \to \mathbb{R}$ for $d \in \{1, 2, \ldots, D\}$ (Li and Chen, 2016) with independent errors ϵ_d, but this optimization still remains quite arduous.

3 Data Description

The image dataset which we use for the coding and simulation aspects of this report is the `Fashion-MNIST` dataset (Xiao et al., 2017), compiled by the software engineering and research team at Zalando. `Fashion-MNIST` is a publicly-available dataset which is commonly used as a benchmark for evaluating the performance of variety of machine learning and statistical tasks. Each of the images in this dataset is a square, 28×28 pixel grayscale image of some clothing item or fashion accessory.

Figure 3.1: Example images from each item category in the `Fashion-MNIST` dataset.

In addition to containing a large collection of images, each of the observations in the dataset is classified into one of ten distinct groups. Figure 3.1 depicts one example from each of these ten item categories. The complete `Fashion-MNIST` dataset contains 70,000 labelled images, with a pre-determined training set of 60,000 images and a test set containing the remaining 10,000 fashion items. The categories are perfectly stratified in both the training and test datasets, with each category comprising of precisely 10% of the total observations in both datasets.

The choice to use the `Fashion-MNIST` dataset is primarily motivated by the simulations in Lawrence (2005). The article by Neil Lawrence – which is one of the foremost papers regarding Gaussian Process Latent Variable Models – uses the 'original' `MNIST` dataset containing 70,000 categorized images of handwritten digits as an example to demonstrate the potential applications of this methodology.

In addition to the relevance to other papers on the topic of dimensionality reduction, the `Fashion-MNIST` dataset is also conveniently made available in many software packages and libraries such as `PyTorch` (Paszke et al., 2019), which makes this benchmark very easy to work with when testing the applicability of different methods.

4 Results

Here, we will discuss the code which we implemented in order to achieve the dimension-reduction goals described in Section 1. All of the code for this project was run using `Python`, with heavy usage of the `numpy`, `PyTorch`, and `scikit-learn` libraries (Harris et al., 2020; Paszke et al., 2019; Pedregosa et al., 2011) for a host of computationally-intensive tasks. The analysis code is split into four main components; Preliminary Analysis, PCA, PPCA, and GP-LVM.

4.1 Preliminary Analyses

Prior to fitting any models, we first loaded the `Fashion-MNIST` dataset into Python and did some brief exploration of the dataset, including looking into examples of each of the classes of fashion items in the dataset, as shown in Figure 3.1.

However, the primary purpose of this stage of the analysis was to determine the value of Q which we would use for dimension reduction. Many papers, such as (Lawrence, 2004), suggest choosing $Q = 2$ or $Q = 3$ for visualization purposes, but we instead decided to use more formal metrics.

Using the designated `Fashion-MNIST` training dataset $\mathbf{Y}^{\text{train}} \in \mathbb{R}^{60000\times784}$, we calculated the variance explained and the proportion of variance explained by each principal component of the data, hoping that this would lead towards an obvious choice for Q. However, as we see in both plots in Figure 4.1, there does not appear to be an obvious 'elbow' where we can cleanly cut off the number of components. Additionally, we found that 49 components are required to explain 80% of the variance in the data, with 136 components needed to surpass the 90% threshold.

In the interest of both time and the safety of my personal computer, we chose to fix $Q = 10$ to be the number of components in the lower-dimensional representations of $\mathbf{Y}^{\text{train}}$. Additionally, due to memory constraints on my computer, we reduced the size of the dataset Y^{train} to 10,000 observations only. This reduction was done in a stratified manner, so that each class of fashion items comprises exactly 10% of the reduced-size dataset, just as it does in the original train and test datasets provided by default when using `Fashion-MNIST`.

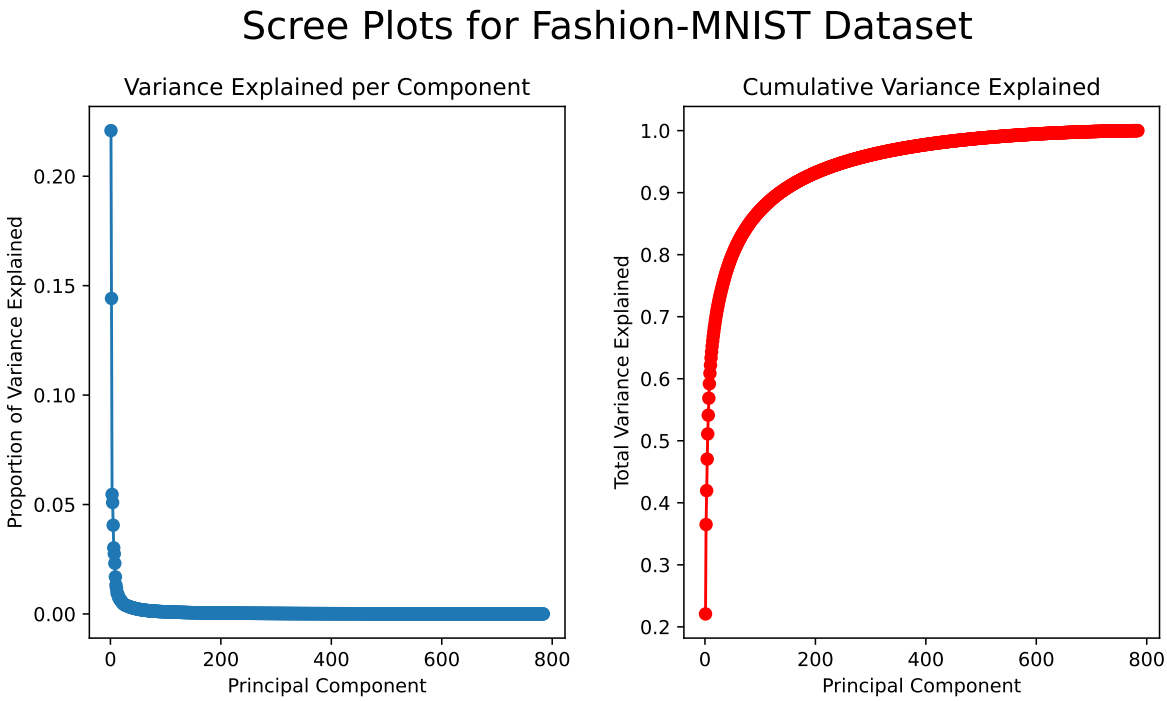
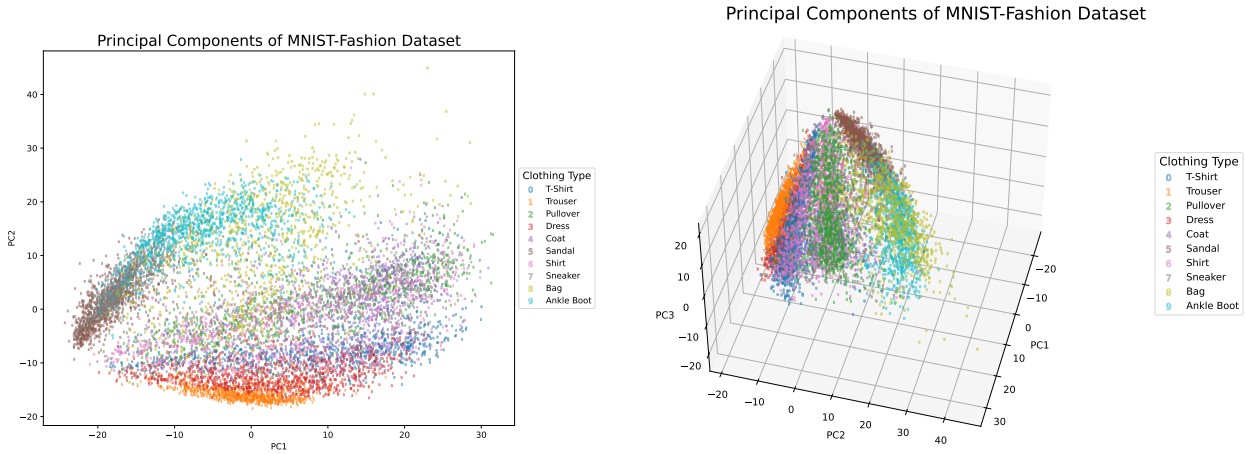


Figure 4.1: Scree plots for the principal components of the training data in the `Fashion-MNIST` dataset

4.2 Principal Component Analysis

Although the 'standard' PCA algorithm is not one of the primary motivations of this report, we chose to use PCA with $Q = 10$ components to perform dimensionality reduction on the sub-sampled dataset.



(a) Top two principal components of clothing items in the `Fashion-MNIST` dataset

(b) Top three principal components of clothing items in the `Fashion-MNIST` dataset

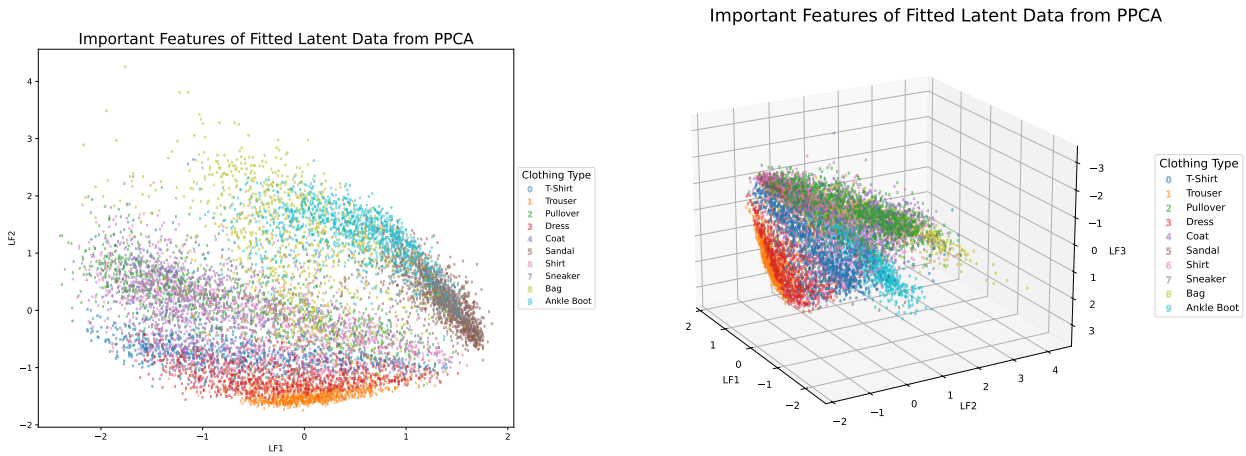
Figure 4.2: Top principal components of clothing items, coloured by clothing type.

Figure 4.2 depicts the top two and top three principal components for the `Fashion-MNIST` dataset, with points coloured based on their corresponding group. With only two components (Figure 4.2a), we see a lot of interspersed data without separation of groups, but with three components (Figure 4.2b), some of the groups which were closely connected seem to spread further from one another. However, it is clear that there are still several fashion items which are not suitably different from one another based on only the first three components, such as trousers and dresses.

After performing the PCA to reduce the dimensionality to 10, we use K-means clustering with 10 clusters to split the lower-dimensional representations of the image data into 10 groups. We then compared these groupings to the true labels of the images using the adjusted Rand index (ARI), which is a metric for comparing groupings in a label-agnostic fashion. The resulting ARI metric was 35.06%, which indicates a subpar performance, but it is better than purely random chance.

4.3 PPCA

Similar to the PCA methodology discussed above, we performed the same set of analytical steps using the exact formulation for PPCA.



(a) Top two latent variables from PPCA for the `Fashion-MNIST` data

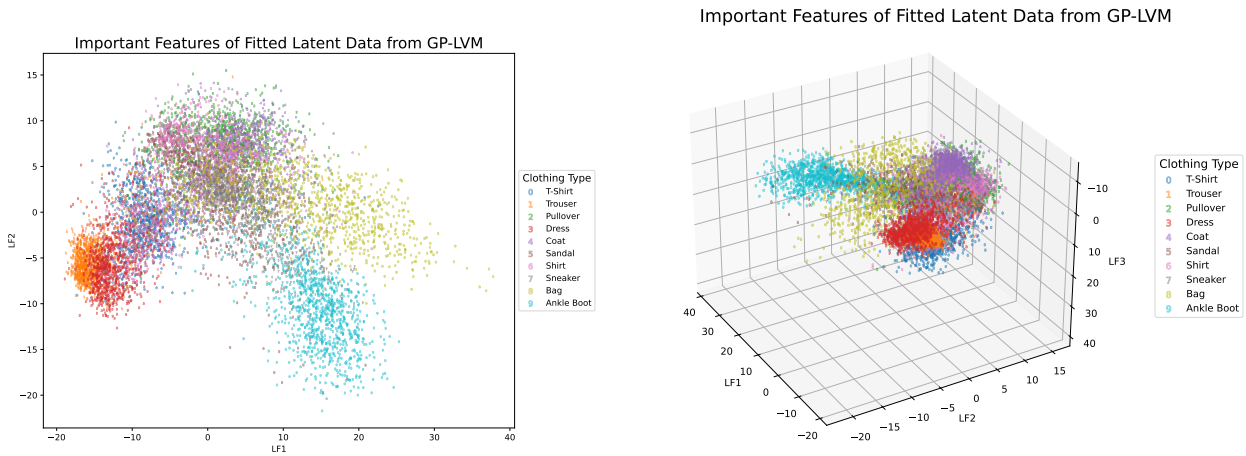
(b) Top three latent variables from PPCA for the `Fashion-MNIST` data

Figure 4.3: Top PPCA latent variables for representing images of clothing items, coloured by clothing type.

Figure 4.3 depicts the top two latent variables for the data using PPCA. Similar to the non-probabilistic PCA we again see that there are several groups which are still stuck together, and there is generally not a lot of separation with only two or three dimensions of latent variables.

Using K-means clustering, we again computed the ARI metric as done for the PCA groupings, and we had a result of 34.63%, which is ever so-slightly worse than the PCA. Additionally, we reconstructed the data from the latent variables, and had a fairly low RMSE, which is quite promising.

4.4 GP-LVM



(a) Top two latent variables from GP-LVM for the `Fashion-MNIST` data

(b) Top three latent variables from GP-LVM for the `Fashion-MNIST` data

Figure 4.4: Highest-importance GP-LVM latent variables for representing images of clothing items, coloured by clothing type.

Figure 4.3 depicts the top two latent variables for the data using PPCA. Similar to the non-probabilistic PCA we again see that there are several groups which are still stuck together, and there is generally not a lot of separation with only two or three dimensions of latent variables.

Using K-means clustering on the resulting latent variable dataset, we computed the ARI metric and had an adjusted Rand index of 38.06%. This is a fair amount higher than the PCA or PPCA results, but this is still not a particularly high number, indicating that there is quite a bit of information for classification that oes not seem to be carried over into the lower-dimensional manifold.

In addition, we reconstructed the dataset based on the latent variable model. However, the RMSE was exactly equal to zero, indicating that the Gaussian Processes are heavily overfitted to the training data, which may be problematic. Unfortunately, running this method for out-of-sample data (such as from $\mathbf{Y}^{\text{test}}$) is quite computationally intensive, so we did not have time to implement this.

Method	Adjusted Rand Index	Reconstruction RMSE
PCA	35.06%	N/A
PPCA	34.63%	0.6114
GPLVM	38.06%	0.0000

Table 4.1: Dimensionality Reduction Results with $Q = 10$.

4.5 Final Comparison

Overall, the GP-LVM was the best in terms of the adjusted Rand index, but none of these methods were particularly incredible when it comes to replicating the true labellings. All three methods performed relatively similarly, which is somewhat disappointing, as the GP-LVM took several hours to run while the PCA was completed in under one minute, and the improvement in performance is not particularly grand.

5 Discussion

5.1 Limitations

One of the primary limitations of this research project is that determining the latent variable matrix $\mathbf{X}^{\text{GPLVM}} \in \mathbb{R}^{N \times Q}$ using the GP-LVM framework is very computationally expensive.

5.2 Future Work

Similar to the main limitation described above, the most important future work would be to potentially re-run this set of simulations with a larger value of Q, so that we could better grasp how well increasing the dimension of the latent variable space will affect the overall results, as these three methods may not all be affected in the same manner.

Additionally, it would be helpful to have the ability to more feasibly perform out-of-sample predictions using the trained models (particularly the GP-LVM), as there appeared to be some severe overfitting in one case, but without out-of-sample data, it is difficult to determine the extent to which this overfitting skewed the results.

References

Harris, C. R., Millman, K. J., der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., and Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Lawrence, N. (2004). Gaussian Process Latent Variable Models for Visualisation of High Dimensional Data. *Advances in Neural Information Processing Systems*, 16.

Lawrence, N. (2005). Probabilistic Non-linear Principal Component Analysis with Gaussian Process Latent Variable Models. *Journal of Machine Learning Research*, 6:1783–1816.

Li, P. and Chen, S. (2016). A review on Gaussian Process Latent Variable Models. *CAAI Transactions on Intelligence Technology*, 1(4):366–376.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Rasmussen, C. E. and Williams, C. K. (2006). *Gaussian Processes for Machine Learning*. The MIT Press.

Tipping, M. E. and Bishop, C. M. (1999). Probabilistic Principal Component Analysis. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 61(3):611–622.

Titsias, M. K. and Lawrence, N. D. (2010). Bayesian gaussian process latent variable model. In Teh, Y. W. and Titterington, M., editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 844–851, Chia Laguna Resort, Sardinia, Italy. PMLR.

Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.